VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

THE OPEN SOURCE AUDIT

A Capstone Project for the Open Source Software Course

Chosen Software:

Python Programming Language

Course	Open Source Software (OSS NGMC)
Unit Coverage	Units 1 – 5
Student Name	Anshita
Registration Number	24BCE11328
Slot	F11
Date of Submission	30 March 2026

1. Introduction

Python is a programming language that I have personally used in basic coding and problem solving. What makes Python different from many other languages is that it is open source and developed by a community rather than a single company.

When I started learning programming, Python felt easier compared to C or C++. The syntax is simple and readable, which made me realize why it became so popular.

In this report, I am not just describing Python as a language, but trying to understand why it was created, how its open-source nature affects it, and how it fits into the larger software ecosystem.

Part A - Origin and Philosophy

A1. The Problem Python Was Created to Solve

Python was created by Guido van Rossum around 1989.

From what I understood, at that time most programming languages had two extremes:

- Either they were powerful but difficult (like C)
- Or they were simple but not useful for larger applications So there was a gap.

Guido wanted to create something that:

- Is easy to read
- Requires less code
- Still powerful enough to build real applications

One thing I noticed is that Python focuses a lot on readability. For example, instead of complicated syntax, it uses indentation to define blocks. This feels more natural, almost like writing structured English.

I think the main problem Python solved was reducing the difficulty of programming. It made coding more accessible, especially for beginners.

Also, Python was shared openly instead of being kept private. I think this decision was important because it allowed people from all over the world to contribute and improve it.

A2. License - What It Actually Says

Python uses something called the PSF (Python Software Foundation) License.

After reading about it, I understood that Python gives users a lot of freedom.

The four main freedoms of open-source software are:

1. You can run the program
2. You can see how it works (source code)
3. You can modify it
4. You can share it

Python satisfies all these freedoms.

One interesting thing is that companies can actually modify Python and use it commercially without sharing their changes. This is because the PSF license is not very restrictive.

This is different from GPL, where if you modify the software, you must also share your changes.

If I compare:

- GPL forces openness
- MIT/PSF allows flexibility

Personally, I think both have their use. If I wanted my project to stay open forever, I would choose GPL. But if I wanted more people to adopt it, I might choose MIT or PSF.

Also, there is this idea of:

- "Free as in free beer" (no cost)
- "Free as in freedom" (control)

Python clearly falls in the second category. It is not just free to use, but also free to modify.

A3. Ethics of Open Source

This part is more about thinking rather than facts.

Should all software be open source?

I don't think the answer is simply yes.

On one hand:

- Open source improves transparency
- Anyone can learn from it
- Bugs can be found faster

But on the other hand:

- Companies need money to survive

- Not all software can be freely shared (like security systems) So I think a balance is needed.

Is it ethical for companies to profit from open source?

This is a bit tricky.

Companies like Google and others use open-source tools like Python and Linux to build products and earn money.

At first, it feels unfair because they didn't build everything themselves.

But then I realized:

- Many of these companies also contribute back
- They improve tools and fund development

So I think it becomes ethical if they contribute back. If they only take and never give, then it feels wrong.

"Standing on the shoulders of giants"

This means building on top of existing work.

In programming, almost everything is built on something else. Python itself depends on older technologies like C and Unix.

So open source doesn't reduce innovation — it actually makes it faster because people don't have to start from zero.

Part B - Linux Footprint

Installation Method

Python is available through the package managers of every major Linux distribution. On Ubuntu and Debian-based systems, installation uses APT:

$ sudo apt install python3 python3-pip

```
root@acer-pc:~# which python
root@acer-pc:~# which python3
/usr/bin/python3
root@acer-pc:~# whereis python3
python3: /usr/bin/python3 /usr/lib/python3 /etc/python3 /usr/share/python3 /usr/share/man/man1/python3.1.gz
root@acer-pc:~#
```

Key Directories on a Linux System

Once installed via the system package manager, Python occupies several standard locations in the Linux filesystem:

Directory	Contents	Purpose
/usr/bin/python3	The interpreter binary	Main executable — run with $ python3
/usr/lib/python3.x/	Standard library modules	All built-in modules: os, sys, re, json, etc.
/usr/lib/python3/distpackages/	System-installed packages	pip-installed packages available system-wide
/etc/python3.x/	System-wide configuration (rare)	Mostly managed through pyproject.toml or pip.conf
~/.local/lib/python3.x/	User-installed packages	Packages installed with pip install --user
/var/log/	No dedicated Python log dir	Python applications write their own logs

User and Group

```
root@acer-pc:~# python3 --version
Python 3.10.12
root@acer-pc:~# python3
Python 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>> exit()
root@acer-pc:~#
```

This matters for security because running as root would give any vulnerability in the application access to the entire system. A dedicated non-privileged user limits the blast radius of a security breach to only the files and permissions that user possesses. **Service Management**

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Python Application
After=network.target

[Service]
Type=simple
User=pyapp
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/server.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

The service is managed with the standard systemctl commands:

```
$ sudo systemctl daemon-reload
$ sudo systemctl start myapp
$ sudo systemctl enable myapp   # Start on boot
$ sudo systemctl status myapp
$ sudo systemctl stop myapp
$ sudo systemctl restart myapp
```

Update Model

Security patches to the Python interpreter itself reach users through two channels: the distribution's package manager (apt upgrade python3 on Ubuntu), and source releases from python.org for users who compiled from source. The Python Security Response Team maintains older Python versions with security-only updates for a defined period.

Part C — The FOSS Ecosystem

Dependencies

CPython, the reference implementation of Python, is written primarily in C. Its compilation depends on GCC or Clang, the GNU C compiler suite. The standard library links against system libraries including libssl (for TLS support), libffi (for calling C code from Python), zlib (for compression), and libbz2. On Linux, these are typically provided by OpenSSL and zlib packages.

Python's build system relies on GNU Make and autoconf. The packaging ecosystem — pip, setuptools, and wheel — is itself written in Python, creating an elegant if occasionally circular dependency structure.

What Python Has Enabled and Inspired

The list of what Python has made possible is genuinely difficult to contain. NumPy and SciPy, released in the 2000s, made Python a viable environment for scientific computing, replacing expensive proprietary tools like MATLAB for many research applications. Django, released in 2005, made Python the language of choice for rapid web development. Flask followed as a microframework for smaller applications.

Forks of the Python interpreter have also produced significant projects. PyPy is a Python interpreter written in Python, using a JIT compiler to achieve substantially better performance than CPython for many workloads. MicroPython strips Python down to run on microcontrollers with kilobytes of RAM, bringing the language to embedded systems.

Python in the Web Stack

Python occupies a well-defined role in web architectures. In the classic LAMP stack (Linux, Apache, MySQL, PHP), Python does not appear — LAMP predates Python's web prominence. But the equivalent Python stack is commonly called LAMP-P or, more accurately, LLMP: Linux, Nginx, MySQL/PostgreSQL, and Python (with Django or Flask as the application layer).

Major web services built on Python include YouTube (which used Python extensively in its early architecture), Instagram (which runs Django at very large scale), Dropbox (which is built substantially in Python), and NASA's web infrastructure. This is not accidental: Python's readability reduces the cost of maintaining large codebases, and its extensive library ecosystem means that most common web development tasks have well-tested solutions available.

Governance and Community

Python is governed by the Python Software Foundation (PSF), a non-profit organisation incorporated in Delaware, USA. The PSF holds the intellectual property rights to the Python programming language, manages the trademark, and funds development, infrastructure, and community events. PyCon, the annual Python conference, is the largest gathering of the Python community and is organised by regional PSF chapters around the world, including PyCon India.

Technical decisions about the language are made through the Python Enhancement Proposal (PEP) process. Any community member can write a PEP proposing a change to the language, the standard library, or the community processes.

Part D — Open Source vs Proprietary Comparison

Comparison with MATLAB

Python's closest proprietary equivalent in the scientific and data analysis domain is MATLAB, developed by MathWorks. The comparison is instructive because MATLAB predates Python's

scientific stack and was, for many years, the standard tool in engineering and research departments worldwide.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free for all users, forever, with no feature tiers	Individual licence: approx. $2,150/year;
Security — code auditability	Full source code available; any researcher can audit CPython and all libraries	Closed source; security depends entirely on MathWorks' internal processes
Support & reliability	Community forums, Stack Overflow, commercial support via third parties (e.g. Anaconda)	Official MathWorks support with guaranteed response times and version stability
Freedom to modify	Complete freedom to fork, modify, and redistribute under the PSF licence	None — users may not reverse-engineer or modify the interpreter
Community vs corporate control	Governed by PSF and an elected Steering Council; contributors worldwide	Controlled entirely by MathWorks; roadmap decided internally
Your overall verdict	Python is the clear choice for reproducible research, education, and general-purpose use. MATLAB retains an edge in specific engineering simulation toolboxes and in	
	industries with deep historical investment in MATLAB codebases.	

Conclusion: Deployment and Contribution

Given everything surveyed in this report, I would recommend Python without hesitation for the vast majority of real-world deployments. Its combination of readability, library ecosystem, community support, and zero licensing cost makes it the rational choice for web applications, data science, automation, and scientific computing. The only context where I would consider MATLAB is one where the existing codebase is already written in MATLAB and migration costs are prohibitive.

As for contribution: Python's PEP process and active GitHub presence make it one of the most accessible large open-source projects to contribute to. Contributions do not have to be modifications to the interpreter itself. Writing documentation, answering questions on discuss.python.org, submitting bug reports with reproducible test cases, or contributing to a single library in the NumPy or Django ecosystem are all meaningful forms of participation. The project exists because thousands of people made this choice. Making it yourself is not altruism — it is an investment in infrastructure that you yourself depend on.

Shell Script Tasks

Script 1 — System Identity Report

This script produces a formatted welcome screen for the Linux system, displaying kernel version, distribution, uptime, current user, date, and the licence that covers the OS. It demonstrates the use of shell variables, command substitution, and formatted output using echo.

```
root@acer-pc:~# #!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: Sachin Parmar | Registration: 24BCE10452
# Course: Open Source Software | VIT Bhopal
# Purpose: Display a formatted summary of the Linux system
# ============================================================

# --- Student and project variables ---
STUDENT_NAME="Sachin Parmar"          # Student's name
SOFTWARE_CHOICE="Python"             # Our chosen open-source software

# --- Collect system information using command substitution ---
KERNEL=$(uname -r)                   # Get the running kernel version
USER_NAME=$(whoami)                  # Get the currently logged-in user
UPTIME=$(uptime -p)                  # Get human-readable system uptime
CURRENT_DATE=$(date '+%A, %d %B %Y') # Get formatted date
CURRENT_TIME=$(date '+%H:%M:%S')     # Get current time

# Detect distro name from /etc/os-release (standard across most distros)
if [ -f /etc/os-release ]; then
    DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d'=' -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

# The OS kernel (Linux) is licensed under GPLv2
OS_LICENSE="GNU General Public License, Version 2 (GPL v2)"

# --- Display the report ---
echo "========================================================"
echo "   Open Source Audit - System Identity Report"
echo "   Student : $STUDENT_NAME"
echo "   Software: $SOFTWARE_CHOICE"
echo "========================================================"
echo ""
echo "  Distribution : $DISTRO"
echo "  Kernel Version: $KERNEL"
echo "  Logged-in User: $USER_NAME"
echo "  Home Directory: $HOME"
echo "  System Uptime : $UPTIME"
echo ""========================================================"
=======================================================
   Open Source Audit - System Identity Report
   Student : Sachin Parmar
   Software: Python
=======================================================

  Distribution : Ubuntu 22.04.3 LTS
  Kernel Version: 6.6.87.2-microsoft-standard-WSL2
  Logged-in User: root
  Home Directory: /root
  System Uptime : up 54 minutes
  Date and Time : Friday, 20 March 2026 at 00:34:33

-------------------------------------------------------
  OS Licence    : GNU General Public License, Version 2 (GPL v2)
  This means the OS source code is freely available,
  modifiable, and redistributable under the same terms.
=======================================================
```

Output :-

```
root@acer-pc:/mnt/c/Users/sachi/Downloads/Programs# ./script1_system_identity.sh
==========================================================
   Open Source Audit — System Identity Report
   Student : Sachin Parmar
   Software: Python
==========================================================

  Distribution : Ubuntu 22.04.3 LTS
  Kernel Version: 6.6.87.2-microsoft-standard-WSL2
  Logged-in User: root
  Home Directory: /root
  System Uptime : up 39 minutes
  Date and Time : Friday, 20 March 2026 at 00:19:18

----------------------------------------------------------
  OS Licence    : GNU General Public License, Version 2 (GPL v2)
  This means the OS source code is freely available,
  modifiable, and redistributable under the same terms.
==========================================================

root@acer-pc:/mnt/c/Users/sachi/Downloads/Programs#
```

Key concepts demonstrated: variables (KERNEL, USER_NAME, DISTRO), command substitution using $(), conditional file checking with [-f], string manipulation using grep, cut, and tr, and formatted output using echo.

Script 2 — FOSS Package Inspector

This script checks whether Python (or any specified package) is installed on the system, retrieves its version and licence metadata, and uses a case statement to print a philosophical note about the package. It demonstrates if-then-else logic, the case statement, package manager queries, and pipes with grep.

```
root@acer-pc:~# #!/bin/bash
# ============================================================
# Script 2: FOSS Package Inspector
# Author: Sachin Parmar | Course: Open Source Software
# Purpose: Check if a FOSS package is installed and print
#          version info and a philosophy note about it.
# Usage  : ./script2_foss_inspector.sh [package_name]
# Example: ./script2_foss_inspector.sh python3
# ============================================================

# Use the first argument if provided, otherwise default to python3
PACKAGE=${1:-python3}

echo "============================================"
echo "  FOSS Package Inspector"
echo "  Checking package: $PACKAGE"
echo "============================================"

# --- Check if the package is installed ---
# Try dpkg first (Debian/Ubuntu), then rpm (RHEL/Fedora)
if dpkg -l "$PACKAGE" &>/dev/null 2>&1; then
    echo "STATUS: $PACKAGE is installed (via dpkg)."
    # Extract and display version information
    dpkg -l "$PACKAGE" | grep "^ii" | awk '{print "Version: " $3}'
    echo ""
    # Show full package details, filtering for useful fields
    dpkg -s "$PACKAGE" 2>/dev/null | grep -E 'Package|Version|Maintainer|Homepage'

elif rpm -q "$PACKAGE" &>/dev/null 2>&1; then
    echo "STATUS: $PACKAGE is installed (via rpm)."
    # Display version, licence and summary from rpm metadata
    rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary|URL'

else
    echo "STATUS: $PACKAGE is NOT installed on this system."
    echo "To install it, try:"
    echo "  Ubuntu/Debian : sudo apt install $PACKAGE"
    echo "  RHEL/Fedora   : sudo dnf install $PACKAGE"
fi

echo ""
echo "--------------------------------------------"
echo "  Philosophy Note:"

# --- Case statement: print a philosophy note per package ---
case $PACKAGE in
    python3|python)
        echo "Python: a language built in the open,"
        echo "shaped entirely by its community, and given"
        echo "freely to the world under the PSF licence."
        ;;
    httpd|apache2)
        echo "Apache: the web server that helped build"
        echo "the open internet, powering ~30% of all sites."
        ;;
    mysql|mariadb)
        echo "MySQL/MariaDB: open-source at the heart of"
        echo "millions of apps. MariaDB was born when the"
        echo "community forked MySQL after Oracle acquired it."
        ;;
    git)
        echo "Git: Linus Torvalds built this after a proprietary"
        echo "tool failed him. Now it underpins all of modern"
        echo "open-source collaboration."
```

```
root@acer-pc:/mnt/c/Users/sachi/Downloads/Programs# ./script2_foss_inspector.sh
============================================
  FOSS Package Inspector
  Checking package: python3
============================================
STATUS: python3 is installed (via dpkg).
Version: 3.10.6-1~22.04.1

Package: python3
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 3.10.6-1~22.04.1
Original-Maintainer: Matthias Klose <doko@debian.org>
Homepage: https://www.python.org/

--------------------------------------------
  Philosophy Note:
Python: a language built in the open,
shaped entirely by its community, and given
freely to the world under the PSF licence.
============================================
```

Script 3 — Disk and Permission Auditor

This script iterates through a list of important Linux system directories and reports the permissions, owner, group, and disk usage of each. It also checks whether Python's sitepackages directory exists. It demonstrates the for loop, directory existence checks with [-d], and the use of ls, du, and awk to extract information.

```
root@acer-pc:~# #!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Author: Sachin Parmar | Course: Open Source Software
# Purpose: Report permissions and sizes of key Linux directories
# ============================================================

# List of important system directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/lib" "/opt")

echo "============================================================"
echo "  Directory Audit Report"
echo "  Generated: $(date '+%Y-%m-%d %H:%M:%S')"
echo "============================================================"
printf "%-25s %-22s %-8s\n" "Directory" "Permissions/Owner" "Size"
echo "------------------------------------------------------------"

# --- Loop through each directory and report its details ---
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, and group using ls -ld and awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3 ":" $4}')
        # Get human-readable disk usage; suppress error for restricted dirs
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-25s %-22s %-8s\n" "$DIR" "$PERMS $OWNER" "${SIZE:-N/A}"
    else
        printf "%-25s %-22s\n" "$DIR" "[Does not exist]"
    fi
done

echo ""
echo "--- Python Installation Check ---"

# Check for Python's site-packages directory (varies by version)
PYTHON_DIRS=("/usr/lib/python3" "/usr/local/lib/python3" "/usr/lib/python3/dist-packages")

for PYDIR in "${PYTHON_DIRS[@]}"; do
    if [ -d "$PYDIR" ]; then
        PYPERMS=$(ls -ld "$PYDIR" | awk '{print $1, $3, $4}')
        echo "Found: $PYDIR"
        echo "       Permissions: $PYPERMS"
    fi
done

# Also check the Python binary location
PYTHON_BIN=$(which python3 2>/dev/null)
if [ -n "$PYTHON_BIN" ]; then
    echo "Python binary : $PYTHON_BIN"
    echo "Binary perms  : $(ls -l $PYTHON_BIN | awk '{print $1, $3, $4}')"
    echo "Python version: $(python3 --version 2>&1)"
else
    echo "python3 binary not found in PATH."
fi

echo "============================================================"
```

Output :-

```
root@acer-pc:/mnt/c/Users/sachi/Downloads/Programs# ./script3_disk_auditor.sh
==========================================================
  Directory Audit Report
  Generated: 2026-03-20 00:24:15
==========================================================
Directory                 Permissions/Owner      Size
----------------------------------------------------------
/etc                      drwxr-xr-x root:root   3.7M
/var/log                  drwxrwxr-x root:syslog 235M
/home                     drwxr-xr-x root:root   4.0K
/usr/bin                  drwxr-xr-x root:root   120M
/tmp                      drwxrwxrwt root:root   24K
/h/usr/lib                  drwxr-xr-x root:root   5.9G
o/opt                      drwxr-xr-x root:root   4.0K

--- Python Installation Check ---
Found: /usr/lib/python3
       Permissions: drwxr-xr-x root root
Found: /usr/lib/python3/dist-packages
       Permissions: drwxr-xr-x root root
Python binary : /usr/bin/python3
Binary perms  : lrwxrwxrwx root root
Python version: Python 3.10.12
==========================================================
```

Script 4 — Log File Analyzer

This script reads a log file line by line, counts occurrences of a specified keyword (defaulting to 'error'), and prints the last five matching lines. It accepts command-line arguments for flexibility. It demonstrates the while-read loop, if-then logic inside loops, counter variables, and command-line argument handling with $1 and $2.

```
root@acer-pc:/mnt/c/Users/sachi/Downloads/Programs# ./script4_log_analyzer.sh /var/log/syslog error
Analysing: /var/log/syslog
Keyword  : error
----------------------------------
Keyword 'error' found 33 time(s) in /var/log/syslog.

Last 5 matching lines:
----------------------------------
Mar 19 23:39:42 acer-pc networkd-dispatcher[158]: ERROR:Unknown state for interface NetworkctlListState(idx=2, name='eth0'
, type='ether', operational='n/a', administrative='unmanaged'): n/a
Mar 19 23:39:49 acer-pc kernel: [    7.918624] WSL (246) ERROR: CheckConnection: getaddrinfo() failed: -5
Mar 19 23:49:57 acer-pc kernel: [  619.707241] platform regulatory.0: Direct firmware load for regulatory.db failed with e
rror -2
Mar 19 23:49:57 acer-pc subiquity_event.406[406]: subiquity/Error/apply_autoinstall_config
Mar 19 23:49:57 acer-pc subiquity_event.406[406]: subiquity/Error/apply_autoinstall_config
----------------------------------
Analysis complete.
```

Script 5 — The Open Source Manifesto Generator

This script interactively asks the user three questions, composes a personalised opensource philosophy statement from their answers, saves it to a timestamped file, and displays it. It demonstrates interactive input with read, string concatenation, file output with >, and the date command. An alias for running the script is shown in a comment.

```
Answer three questions. Your manifesto will be generated.

1. Name one open-source tool you use every day: VLC Media Player
2. In one word, what does 'freedom' mean to you?  to not be a slave to an idea, concept And person
3. Name one thing you would build and share freely: greatness

Manifesto saved to: manifesto_root_20260320_003251.txt

=======================================================
          OPEN SOURCE MANIFESTO
          Generated by: root
          Date: 20 March 2026
=======================================================

Every day, I rely on VLC Media Player — a piece of software that
someone built, shared, and gave to the world without
asking for anything in return. To me, freedom means
to not be a slave to an idea, concept And person.

I believe that code, like knowledge, grows stronger when
it is shared. That is why I would build greatness and
release it openly — not because I am required to, but
because the software I depend on every day was built by
people who made the same choice before me.

Open source is not just a licensing model. It is an
argument about how knowledge should move through the
world.

                    — root, 20 March 2026
=======================================================
```